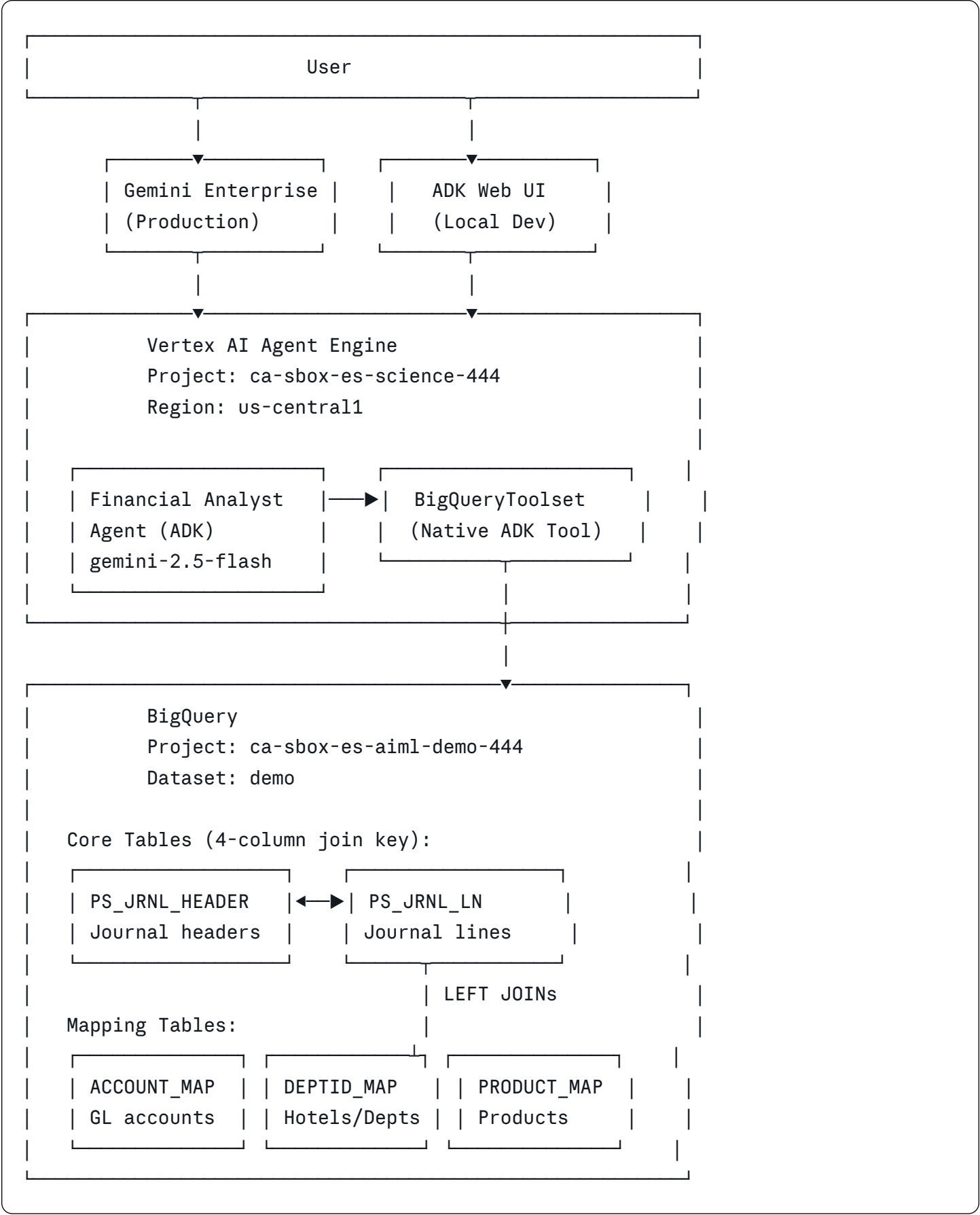


Financial Analyst Agent

An AI-powered financial analysis agent built with Google's Agent Development Kit (ADK) that queries accounting journal data in BigQuery and provides actionable financial insights.

Architecture



How It Works

1. User asks a financial question in natural language.
2. The agent generates an optimized BigQuery SQL query based on the table schemas baked into its instructions.
3. BigQueryToolset executes the query against the cross-project dataset.
4. The agent returns results with the SQL query, key insights, and 3 follow-up questions.

Data Model

Core Tables

PS_JRNL_HEADER — Journal entry headers with metadata (dates, totals, status).

PS_JRNL_LN — Journal line items with account details, amounts, and descriptions.

These tables join on 4 columns: `BUSINESS_UNIT`, `JOURNAL_ID`, `JOURNAL_DATE`, `UNPOST_SEQ`.

Mapping Tables

Mapping Table	Joins On	Provides
ACCOUNT_MAP_AGENT_DATA	PS_ACCOUNT → ACCOUNT	Account names, types, groups
DEPTID_MAP_AGENT_DATA	DEPTID → DEPTID	Hotel names, brands, regions, countries
PRODUCT_CODE_MAP_AGENT_DATA	PRODUCT → PRODUCT	Product names, lines, business segments

Prerequisites

- Python 3.11+
- Google Cloud SDK (`gcloud` CLI)
- GCP project with BigQuery and Vertex AI access
- Application Default Credentials (ADC) or service account

IAM Roles Required

Deploy project (ca-sbox-es-science-444):

- `roles/aiplatform.user` — deploy and run agents
- `roles/storage.objectAdmin` — staging bucket access

Data project (ca-sbox-es-aiml-demo-444):

- `roles/bigquery.dataViewer` — read BigQuery tables
- `roles/bigquery.jobUser` — execute queries

Quick Start

1. Clone and Install

```
bash

git clone https://bitbucket.ihgint.global/scm/dst/mlops_internal.git
cd mlops_internal/finance_agent
pip install -r requirements.txt
```

2. Configure Environment

```
bash

cp .env.example .env
```

Edit `.env` with your values:

```
dotenv

GCP_PROJECT_ID=ca-sbox-es-science-444
GCP_REGION=us-central1
STAGING_BUCKET=gs://ca-sbox-es-science-444
BQ_PROJECT_ID=ca-sbox-es-aiml-demo-444
BQ_DATASET=demo
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GEMINI_MODEL=gemini-2.5-flash
AGENT_ENGINE_RESOURCE=
```

3. Authenticate

```
bash

gcloud auth application-default login
```

4. Test Locally

```
bash

# From the parent directory of finance_agent/
cd ..
adk web
```

Open `http://localhost:8000`, select **finance_agent** in the dropdown, and start chatting.

Sample Queries

- "Show me total debits and credits for last month"
- "What are the top 10 expense accounts by amount?"
- "Show revenue by hotel brand for January 2026"
- "Break down transactions by hotel region"
- "Which products have the highest revenue?"
- "Find all journal entries with amounts over \$1M"
- "Compare debits vs credits by accounting period"

Deployment

Deploy to Agent Engine (New)

```
bash

adk deploy agent_engine \
  --project=ca-sbox-es-science-444 \
  --region=us-central1 \
  --display_name="Financial Analyst" \
  --description="Analyzes accounting journal transactions with insights" \
  --trace_to_cloud \
  finance_agent/
```

Update Existing Deployment

```
bash

adk deploy agent_engine \
  --project=ca-sbox-es-science-444 \
  --region=us-central1 \
  --display_name="Financial Analyst" \
  --description="Analyzes accounting journal transactions with insights" \
  --trace_to_cloud \
  --agent_engine_id=<REASONING_ENGINE_ID> \
  finance_agent/
```

Add Labels

```
bash
```

```
gcloud ai reasoning-engines update <REASONING_ENGINE_ID> \
  --project=ca-sbox-es-science-444 \
  --region=us-central1 \
  --update-labels=env=dev,team=aiml,project=finance,data-project=aiml-demo
```

Register with Gemini Enterprise

1. Go to GCP Console → Gemini Enterprise → your app → Agents.
2. Click **Add agent** → **Custom agent via Agent Engine**.
3. Enter the Agent Engine ID and set location to `us-central1`.
4. Grant the Gemini Enterprise service account BigQuery access on the data project.

File Structure

```
finance_agent/
├── __init__.py          # Exports root_agent for ADK discovery
├── agent.py             # Agent definition with system prompt and schema
├── config.py            # Configuration from environment variables
├── deploy.py            # Legacy deployment script (use adk deploy instead)
├── session_runner.py    # Interactive CLI testing tool
├── requirements.txt     # Python dependencies
├── .env                 # Environment variables (not committed)
├── .env.example         # Environment variable template
├── .adk/
│   └── settings.json    # ADK Web UI configuration
└── tools/
    ├── __init__.py
    └── bigquery_tool.py  # BigQueryToolset wrapper
```

Response Format

Every response from the agent includes 4 sections:

1. **SQL Query** — the exact query executed, in a code block for transparency
2. **Data Results** — formatted table with human-readable names
3. **Key Insights** — 2-4 actionable financial observations
4. **Follow-Up Questions** — 3 suggested questions to dig deeper

Configuration

Variable	Default	Purpose
GCP_PROJECT_ID	ca-sbox-es-science-444	Agent Engine deploy project
GCP_REGION	us-central1	Region for Agent Engine
STAGING_BUCKET	gs://ca-sbox-es-science-444	GCS bucket for deployment
BQ_PROJECT_ID	ca-sbox-es-aiml-demo-444	BigQuery data project
BQ_DATASET	demo	BigQuery dataset name
GEMINI_MODEL	gemini-2.5-flash	LLM model
GOOGLE_GENAI_USE_VERTEXAI	TRUE	Use Vertex AI (not API key)
AGENT_ENGINE_RESOURCE	(empty)	Set after deployment

Troubleshooting

"No API key was provided" — Add `GOOGLE_GENAI_USE_VERTEXAI=TRUE` to your `.env` file.

Agent generates SQL but doesn't execute it — The Agent Engine service account lacks BigQuery permissions. Grant `bigquery.dataViewer` and `bigquery.jobUser` on the data project.

"Multiple tools are supported only when they are all search tools" — This occurs when mixing built-in tools (code execution) with function tools. Use only function tools or separate into sub-agents.

Deployment times out (DEADLINE_EXCEEDED) — Retry the deployment. If persistent, deploy as a new agent and delete the old one.

ADK Web shows "tools" instead of agent name — Run `adk web` from the parent directory of `finance_agent/`, not from inside it.

Import errors with relative imports — The `agent.py` uses try/except for both package mode (`adk web`) and direct execution (`python session_runner.py`).

Security Notes

1. Never commit `.env` or service account key files to version control.
2. The agent executes read-only SQL queries (SELECT only).
3. For production, use Workload Identity Federation instead of key files.
4. Agent Engine deployments are private to your GCP project.
5. When registered in Gemini Enterprise, access is controlled via IAM.

Team

- **Team:** AIML
- **Project:** Finance
- **Environment:** Dev
- **Data Project:** aiml-demo